Настоящее учебно-методическое пособие предназначено для студентов высших учебных заведений, изучающих дисциплину «Базы данных». Пособие сохраняет фундаментальные темы курса — проектирование, нормализацию, язык SQL, обеспечение целостности — и одновременно приближает практикум к современным требованиям отрасли. В качестве сквозного учебного кейса используется разработка базы данных маркетплейса (аналог Ozon, Wildberries, Яндекс Маркет), что позволяет изучать технологии на задачах, близких к реальной разработке информационных систем. В качестве СУБД применяется PostgreSQL.

Пособие состоит из общих теоретических разделов и семи практических работ: от анализа предметной области и проектирования схемы до оконной аналитики, серверной логики, работы с JSONB и оптимизации запросов. Каждая лабораторная работа содержит развёрнутую теоретическую часть, достаточную для выполнения задания и ответа на контрольные вопросы, техническое задание на основе сквозного кейса и контрольные вопросы для самопроверки и подготовки к защите.

Ключевые слова: базы данных, PostgreSQL, SQL, ER-моделирование, нормализация, JOIN, оконные функции, транзакции, ACID, PL/pgSQL, триггеры, индексы, EXPLAIN, JSONB, маркетплейс.

Содержание

Содержание 2
Введение 5
Сквозной кейс: предметная область маркетплейса 7
Инструменты разработки 9
Система контроля версий 11
Требования к выполнению лабораторных работ 12
Лабораторная работа №1 13
Проектирование и создание структуры базы данных 13
Теоретическая часть 13
Реляционная модель данных 13
Этапы проектирования и ER-модель 13
Ключи 14
Связи и ссылочная целостность 14
Ограничения целостности 15
Нормализация 15
Иерархические данные и реализация схемы 16
Техническое задание 17
Контрольные вопросы 18
Лабораторная работа №2 19
Базовые SQL-запросы и фильтрация данных 19
Теоретическая часть 19
Структура команды SELECT 19
Фильтрация строк: WHERE и предикаты 19
Значение NULL и трёхзначная логика 20
Сортировка, ограничение, уникальность 20
Наполнение таблиц данными 20
Техническое задание 21
Контрольные вопросы 21
Лабораторная работа №3 23
Многотабличные запросы и аналитические представления 23
Теоретическая часть 23
Соединение таблиц (JOIN) 23

Агрегатные функции и группировка 23
Подзапросы 24
Общие табличные выражения (CTE) 24
Представления (VIEW) 25
Техническое задание 25
Контрольные вопросы 26
Лабораторная работа №4 27
Оконные функции и аналитика данных 27
Теоретическая часть 27
Понятие оконной функции 27
Синтаксис: конструкция OVER 27
Ранжирующие функции 27
Агрегатные функции в роли оконных и функции смещения 28
Топ-N в группе и сравнение с GROUP BY 28
Техническое задание 28
Контрольные вопросы 29
Лабораторная работа №5 30
Транзакции, целостность данных и серверная логика 30
Теоретическая часть 30
Транзакции и свойства ACID 30
Функции и язык PL/pgSQL 31
Триггеры 31
Логика в базе данных или в приложении 32
Техническое задание 32
Контрольные вопросы 33
Лабораторная работа №6 35
Современные возможности PostgreSQL и оптимизация запросов 35
Теоретическая часть 35
Тип данных JSONB 35
Как СУБД исполняет запрос 35
Индексы 36
Анализ плана: EXPLAIN и EXPLAIN ANALYZE 36
Цена индексов и GIN 36

Техническое задание 37
Контрольные вопросы 38
Лабораторная работа №7 39
Итоговый проект 39
Общее описание 39
Состав работы 39
Критерии оценивания 39

Введение

База данных — это организованная совокупность структурированных данных, хранимых и управляемых так, чтобы их можно было эффективно сохранять, изменять и извлекать. Системой управления базами данных (СУБД) называют программный комплекс, который обеспечивает создание базы данных, целостность хранимых данных, многопользовательский доступ и выполнение запросов. Умение проектировать базы данных и работать с ними — одна из базовых компетенций разработчика информационных систем: практически любое приложение, от мобильного клиента до крупного маркетплейса, опирается на надёжное хранилище данных.

В основе курса лежит реляционная модель данных и язык SQL. Реляционная модель описывает данные в виде набора связанных таблиц с чётко заданной структурой и правилами целостности, а SQL предоставляет декларативный способ описывать, какие данные нужно получить или изменить, не указывая, как именно СУБД должна это сделать. Такой подход остаётся отраслевым стандартом и лежит в основе большинства промышленных систем.

В качестве СУБД используется PostgreSQL — свободная объектно-реляционная система, сочетающая строгое следование стандарту SQL с богатым набором современных возможностей: оконными функциями, общими табличными выражениями (CTE), многоверсионным управлением конкурентным доступом (MVCC), процедурным языком PL/pgSQL, типом JSONB и развитой системой индексов. Этот набор позволяет в рамках одного инструмента пройти путь от простых выборок до серверной аналитики и оптимизации производительности.

Сквозным учебным кейсом служит база данных маркетплейса — площадки, на которой продавцы размещают товары, а покупатели оформляют заказы. Все семь работ развивают одну и ту же базу данных: спроектированная в первой работе схема последовательно наполняется данными, обрастает аналитическими запросами, серверной логикой и оптимизируется по производительности.

По результатам освоения курса студент должен уметь:

— проектировать структуры баз данных на основе анализа предметной области;

—создавать и настраивать реляционные базы данных средствами PostgreSQL;

—разрабатывать SQL-запросы различной сложности, включая многотабличные и аналитические выборки;

—использовать современные возможности PostgreSQL: оконные функции, CTE, JSONB;

—обеспечивать целостность данных декларативными и процедурными средствами;

—реализовывать бизнес-логику на стороне СУБД — функции и триггеры на языке PL/pgSQL;

—оптимизировать производительность запросов с помощью индексов и анализа планов выполнения;

—разрабатывать и защищать законченный проект базы данных для реального цифрового продукта.

Сквозной кейс: предметная область маркетплейса

Маркетплейс — электронная площадка, объединяющая множество независимых продавцов и покупателей. Продавцы размещают товары в каталоге, покупатели находят их, добавляют в корзину и оформляют заказы, после чего товары оплачиваются и доставляются. На протяжении всех работ разрабатывается база данных, обслуживающая такую площадку.

В предметной области выделяются следующие основные сущности, которые станут таблицами проектируемой базы данных:

—Пользователи (users) — учётные записи площадки; пользователь может быть покупателем, а также владельцем магазина-продавца.

—Продавцы (sellers) — магазины, размещающие товары; связаны с пользователем-владельцем.

—Категории (categories) — иерархический классификатор товаров (категория может иметь родительскую категорию).

—Товары (products) — карточки товаров, принадлежащие продавцу и отнесённые к категории.

—Складские остатки (stock) — количество доступного к продаже товара.

—Корзины (carts) и позиции корзины (cart_items) — отложенные к покупке товары.

—Заказы (orders) и позиции заказа (order_items) — оформленные покупки; связь товаров и заказов «многие-ко-многим».

—Платежи (payments) — сведения об оплате заказов.

—Отзывы (reviews) — оценки и комментарии покупателей к товарам.

—Адреса (addresses) и доставки (deliveries) — куда и как доставляется заказ.

Эта предметная область сознательно богата связями всех видов: «один-ко-многим» (продавец — товары), «многие-ко-многим» (заказы — товары через позиции), иерархия (категории), что делает её удобной основой для изучения проектирования, многотабличных запросов и аналитики.

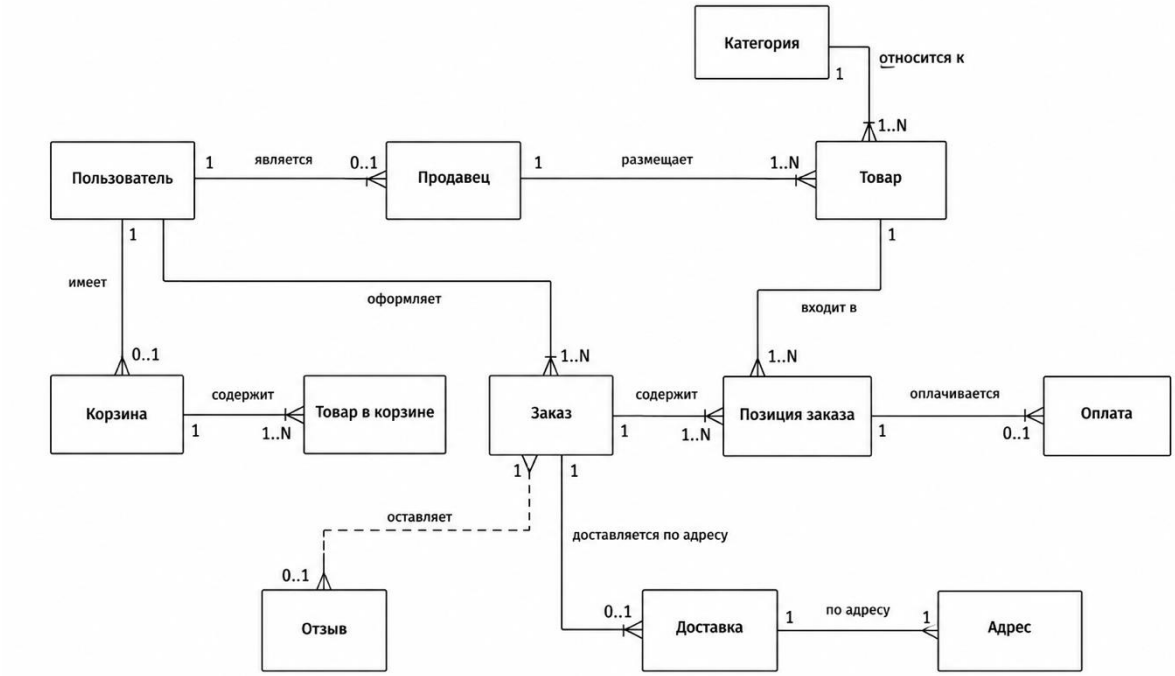


Рисунок 1 — Концептуальная ER-модель предметной области маркетплейса

Инструменты разработки

PostgreSQL — свободная объектно-реляционная СУБД с открытым исходным кодом, развиваемая сообществом более тридцати лет. Она доступна на всех основных операционных системах, строго следует стандарту SQL и поддерживает расширенные возможности, изучаемые в курсе. Для выполнения работ достаточно локально установить сервер PostgreSQL и один из клиентов для подключения и выполнения запросов.

psql — официальный клиент командной строки, входящий в поставку PostgreSQL. Он позволяет подключаться к базе, выполнять SQL-команды и служебные мета-команды: \l — список баз, \dt — список таблиц, \d имя_таблицы — структура таблицы, \timing — измерение времени выполнения. psql незаменим для скриптов и автоматизации.

Для повседневной работы удобны графические клиенты. pgAdmin — официальное веб-приложение для администрирования PostgreSQL с деревом объектов, редактором запросов и средствами просмотра планов выполнения. DBeaver — универсальный клиент с автодополнением и визуализацией данных. Выбор клиента не влияет на содержание работ: все задания выполняются стандартным SQL.

Типовой порядок начала работы: установить сервер PostgreSQL; создать учебную базу командой CREATE DATABASE marketplace; подключиться к ней; выполнять SQL-скрипты работ. Рекомендуется оформлять все команды в виде сохраняемых скриптов (.sql), чтобы базу можно было воссоздать с нуля и воспроизвести результаты.

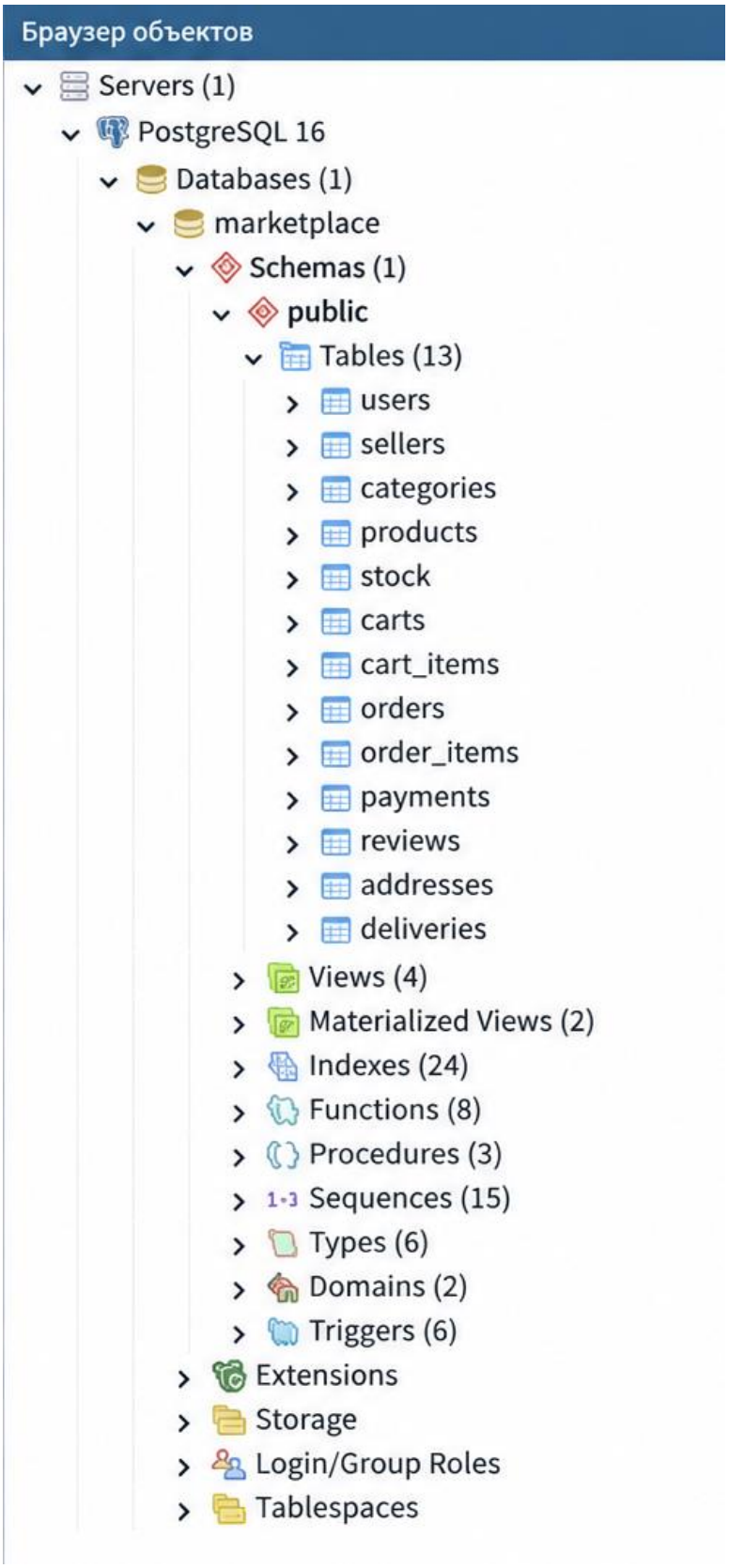


Рисунок 2 — Дерево объектов базы данных в графическом клиенте

Система контроля версий

Git — распределённая система контроля версий, предназначенная для ведения истории изменений в файлах проекта. Она фиксирует состояние проекта во времени и позволяет отслеживать, кто, когда и какие изменения внёс. При выполнении работ Git используется для хранения SQL-скриптов, отчётов и итогового проекта, а также для сдачи работ преподавателю.

В базовом цикле применяются три команды. git add добавляет изменённые файлы в индекс (помечает, что попадёт в фиксацию). git commit сохраняет подготовленные изменения в локальной истории в виде коммита с поясняющим сообщением. git push отправляет локальные коммиты в удалённый репозиторий.

Ветвление позволяет вести разработку параллельными линиями. Под каждую работу удобно создавать отдельную ветку (например, lab_1), а по завершении объединять её с основной через pull request на платформе хостинга (GitHub, GitLab). Pull request фиксирует предлагаемые изменения, позволяет провести их проверку и затем выполнить слияние.

Требования к выполнению лабораторных работ

1. Все работы выполняются в СУБД PostgreSQL (рекомендуемая версия — 15 или новее).
2. Результат каждой работы оформляется в виде SQL-скрипта (скриптов), который можно выполнить повторно и воспроизвести результат.
3. Все объекты базы данных именуются осмысленно, на латинице, в едином стиле (рекомендуется snake_case).
4. К работе прилагается краткий отчёт: постановка задачи, принятые решения и их обоснование, тексты ключевых запросов и результаты их выполнения.
5. Скрипты и отчёт размещаются в репозитории системы контроля версий; каждая работа — в отдельной ветке или папке.
6. Сквозная база данных развивается от работы к работе: схема, созданная в первой работе, используется во всех последующих.
7. Защита работы предполагает не только демонстрацию результата, но и объяснение принятых решений и ответы на контрольные вопросы.

Лабораторная работа №1

Проектирование и создание структуры базы данных

Теоретическая часть

Реляционная модель данных

Реляционная модель, предложенная Э. Коддом, представляет данные в виде отношений. Отношение на практике — это таблица, состоящая из строк и столбцов. Строку называют также кортежем или записью: она описывает один объект предметной области, например один товар. Столбец называют атрибутом или полем: он описывает одно свойство объекта и связан с доменом — множеством допустимых значений, которое в СУБД задаётся типом данных и ограничениями. Совокупность имён столбцов и их типов образует схему отношения; она фиксируется при создании таблицы и определяет, какие данные допустимо в ней хранить. Важные свойства отношения: строки не упорядочены (порядок задаётся только запросом), а каждое значение атомарно — в одной ячейке хранится одно неделимое значение.

Этапы проектирования и ER-модель

Проектирование базы данных проходит через три уровня моделей. Концептуальная модель описывает сущности и связи между ними без привязки к конкретной СУБД и обычно изображается ER-диаграммой (Entity — Relationship, «сущность — связь»). Логическая модель уточняет атрибуты каждой сущности, первичные и внешние ключи, типы связей. Физическая модель — это конкретные таблицы PostgreSQL с типами данных и ограничениями, готовые к созданию командами DDL. Такое разделение позволяет сначала договориться о составе данных, не отвлекаясь на технические детали, и лишь затем перейти к реализации.

На ER-диаграмме сущность — это объект, о котором хранят данные (товар, заказ, пользователь); атрибут — его свойство (наименование, цена); связь отражает отношение между сущностями. Для связи указывают кардинальность — сколько объектов одной сущности соответствует объекту другой. Различают три вида связей. «Один-к-одному» (1:1) встречается редко и обычно выносит редкие или чувствительные атрибуты в отдельную таблицу. «Один-ко-многим» (1:N) — самый частый вид: одному продавцу принадлежит много товаров. «Многие-ко-многим» (M:N) означает, что объектам с обеих сторон соответствует множество объектов

другой стороны: один заказ содержит много товаров, а один товар входит во многие заказы.

Ключи

Каждая строка таблицы должна однозначно идентифицироваться. За это отвечает первичный ключ (PRIMARY KEY) — столбец или набор столбцов, значения которых уникальны и не равны NULL. Требования к первичному ключу: уникальность (не бывает двух строк с одинаковым ключом), обязательность (значение всегда задано), стабильность (значение не должно меняться со временем). Если уникальных столбцов несколько, их называют потенциальными (candidate) ключами; один из них выбирают первичным, остальные остаются альтернативными и поддерживаются ограничением UNIQUE.

В качестве первичного ключа используют суррогатный или естественный ключ. Суррогатный ключ — искусственный идентификатор, не несущий смысла предметной области (целочисленный автоинкремент или UUID). Его достоинства: стабильность (он никогда не меняется), компактность, независимость от изменений бизнес-правил. Недостаток — он не нагляден и требует соединения с другими таблицами, чтобы понять смысл строки. Естественный ключ образован из реальных атрибутов (например, артикул товара или email пользователя). Он нагляден и иногда уже уникален по смыслу, но менее устойчив: реальные данные имеют свойство меняться и иногда оказываются не такими уникальными, как предполагалось. В учебной схеме рекомендуется суррогатный ключ, объявляемый как GENERATED ALWAYS AS IDENTITY.

Связи и ссылочная целостность

Связи между таблицами реализуются внешними ключами (FOREIGN KEY). Внешний ключ — столбец, значения которого ссылаются на первичный ключ другой (или той же) таблицы. Например, в таблице товаров столбец seller_id ссылается на первичный ключ таблицы продавцов. Внешний ключ обеспечивает ссылочную целостность: СУБД не позволит вставить товар с несуществующим продавцом и контролирует судьбу зависимых строк при изменении или удалении родительской.

Поведение при удалении или изменении родительской строки задаётся правилами ON DELETE и ON UPDATE. ON DELETE RESTRICT (и близкое NO ACTION) запрещает удаление родителя, пока на него ссылаются дочерние строки. ON DELETE CASCADE удаляет дочерние строки

автоматически вслед за родителем. ON DELETE SET NULL обнуляет внешний ключ в дочерних строках, ON DELETE SET DEFAULT — подставляет значение по умолчанию. Выбор правила — проектное решение: удаление товара не должно молча удалять позиции уже оформленных заказов (здесь уместен RESTRICT), а удаление заказа логично каскадно удаляет его позиции (CASCADE). Связь «многие-ко-многим» реализуется ассоциативной таблицей с двумя внешними ключами; её первичным ключом обычно служит пара этих ключей.

Ограничения целостности

Целостность данных бывает доменной (значение принадлежит допустимому множеству), сущностной (каждая строка уникально идентифицируется первичным ключом) и ссылочной (внешние ключи указывают на существующие строки). PostgreSQL поддерживает декларативные ограничения, которые задаются один раз в схеме и автоматически проверяются при любых операциях:

—PRIMARY KEY — первичный ключ (уникальность и NOT NULL одновременно);

—FOREIGN KEY — внешний ключ (ссылочная целостность);

—NOT NULL — запрет отсутствия значения;

—UNIQUE — уникальность значений столбца (например, email пользователя);

—CHECK — произвольное логическое условие на значение (например, price > 0);

—DEFAULT — значение по умолчанию при отсутствии явного.

Нормализация

Нормализация — процесс приведения схемы к виду, при котором сокращается избыточность и устраняются аномалии: вставки (нельзя добавить данные об объекте, пока нет связанного с ним), обновления (одно и то же значение приходится менять во многих строках, рискуя рассогласовать данные) и удаления (удаление одной строки теряет полезные сведения о другом объекте). Источник аномалий — смешение в одной таблице независимых сущностей и наличие нежелательных функциональных зависимостей между атрибутами.

Первая нормальная форма (1НФ) требует атомарности значений: никаких списков и повторяющихся групп в одной ячейке. Вторая

нормальная форма (2НФ) требует, чтобы каждый неключевой атрибут зависел от всего составного первичного ключа, а не от его части (актуально только при составном ключе). Третья нормальная форма (3НФ) дополнительно запрещает транзитивные зависимости — когда неключевой атрибут зависит от другого неключевого. Нормальная форма Бойса — Кодда (БКНФ) усиливает 3НФ, требуя, чтобы любая функциональная зависимость исходила из потенциального ключа. На практике корректная схема в 3НФ устраняет большинство аномалий и считается базовым целевым уровнем. Иногда ради производительности применяют осознанную денормализацию — намеренно вводят избыточность (например, хранят заранее вычисленный рейтинг товара), понимая, что взамен придётся поддерживать согласованность дублированных данных.

Иерархические данные и реализация схемы

Иерархию (дерево категорий) моделируют ссылкой таблицы на саму себя: столбец parent_id является внешним ключом на первичный ключ той же таблицы. Корневые категории имеют parent_id, равный NULL. Такой способ называют списком смежности; для обхода всего поддерева применяют рекурсивные запросы (рассматриваются в работе №3). Пример определения таблицы категорий с самоссылкой, таблицы товаров и ассоциативной таблицы позиций заказа:

```
CREATE TABLE categories (
    id        BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    parent_id BIGINT REFERENCES categories(id) ON DELETE RESTRICT,
    name      TEXT NOT NULL UNIQUE
);

CREATE TABLE products (
    id          BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    seller_id   BIGINT NOT NULL REFERENCES sellers(id) ON DELETE RESTRICT,
    category_id BIGINT NOT NULL REFERENCES categories(id),
    title       TEXT NOT NULL,
    price       NUMERIC(12,2) NOT NULL CHECK (price > 0),
    is_active   BOOLEAN NOT NULL DEFAULT TRUE,
    created_at  TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE order_items (
    order_id   BIGINT NOT NULL REFERENCES orders(id) ON DELETE CASCADE,
    product_id BIGINT NOT NULL REFERENCES products(id) ON DELETE RESTRICT,
    quantity   INTEGER NOT NULL CHECK (quantity > 0),
    unit_price NUMERIC(12,2) NOT NULL CHECK (unit_price > 0),
    PRIMARY KEY (order_id, product_id)
);
```

PostgreSQL предоставляет богатый набор типов данных: целочисленные (INTEGER, BIGINT), числа с фиксированной точностью (NUMERIC) для денежных величин, строковые (TEXT, VARCHAR), логический (BOOLEAN), дата и время (DATE, TIMESTAMPTZ — момент времени с часовым поясом), а также UUID, массивы и JSONB. Цену хранят в NUMERIC, а не в типах с плавающей точкой, чтобы избежать ошибок округления денежных сумм.

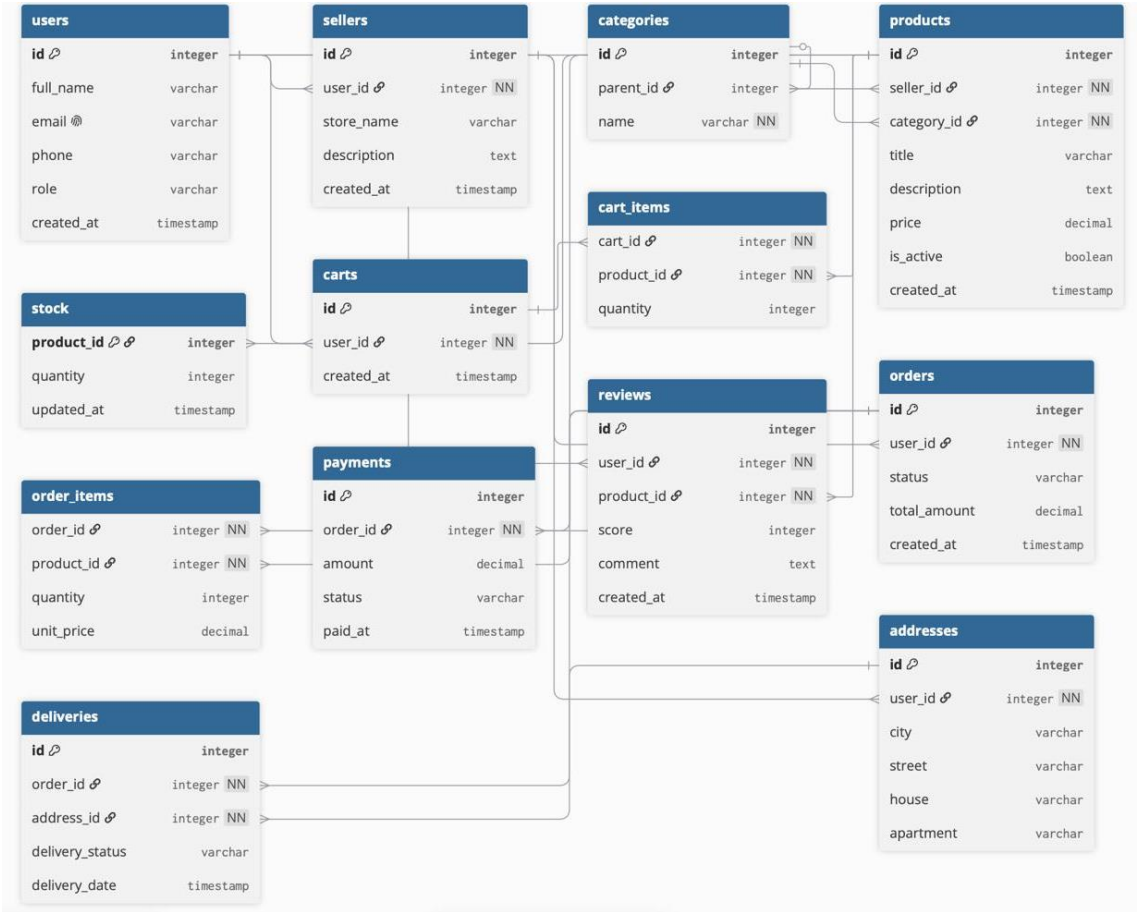


Рисунок 3 — Логическая схема базы данных (диаграмма «сущность — связь»)

Техническое задание

Спроектируйте и реализуйте структуру базы данных маркетплейса согласно описанию предметной области.

1. Проведите анализ предметной области и сформулируйте перечень бизнес-правил (например: у товара ровно один продавец; заказ принадлежит одному покупателю и содержит хотя бы одну позицию; отзыв оставляется к конкретному товару конкретным пользователем).
2. Постройте ER-диаграмму, включающую не менее десяти сущностей. Схема должна содержать хотя бы одну связь «многие-ко-многим»

через ассоциативную таблицу и хотя бы одну иерархическую связь (самоссылку).

3. Приведите схему к третьей нормальной форме. На примере одной таблицы покажите аномалию до нормализации и поясните, как она устранена.
4. Определите для каждой таблицы первичный ключ; обоснуйте выбор суррогатных или естественных ключей.
5. Реализуйте схему в PostgreSQL средствами DDL: создайте таблицы с корректными типами, первичными и внешними ключами, ограничениями NOT NULL, UNIQUE, CHECK, DEFAULT и правилами ON DELETE / ON UPDATE.
6. Составьте словарь данных: для каждой таблицы перечислите столбцы, их типы, ограничения и назначение.
7. Оформите все команды единым SQL-скриптом, повторное выполнение которого воссоздаёт схему с нуля.

Контрольные вопросы

1. Что такое реляционная модель данных и что в ней называют отношением, кортежем и атрибутом?
2. Чем отличаются концептуальная, логическая и физическая модели данных?
3. Что такое первичный ключ и какие требования к нему предъявляются?
4. В чём различие между суррогатным и естественным ключом? Каковы их достоинства и недостатки?
5. Что такое внешний ключ и как он обеспечивает ссылочную целостность?
6. Как в реляционной модели реализуется связь «многие-ко-многим»?
7. Как смоделировать иерархические данные (дерево категорий) одной таблицей?
8. Какие ограничения целостности поддерживает PostgreSQL и что задаёт каждое из них?
9. Что такое нормализация и какие аномалии она устраняет? Сформулируйте требования 1НФ, 2НФ и 3НФ.
10. Для чего нужны правила ON DELETE RESTRICT и ON DELETE CASCADE и чем они различаются?